Linear Regression & Regularization

AI Accelerator Program

Session 10 | 4:30 PM – 6:30

Instructor;
Mahi Aminu Aliyu
AI & SI Associate AHFID

Today's Learning Outcomes

By the end of this session, you should understand:

- The historical origin of regression (Galton's insight)
- When and why models underfit or overfit
- How regularization controls model complexity
- The train/test split mental model
- Practical pipeline patterns for Nigerian housing data

Session Flow

- **Part 1:** The origin story (Galton 1886)
- **Part 2:** Linear regression fundamentals
- **Part 3:** Polynomial features & overfitting
- **Part 4:** Regularization (Ridge/Lasso/ElasticNet)
- **Part 5:** Cross-validation & hyperparameter tuning
- **Practice + Assignment:** hands-on notebooks

Part 1 — The Origin of "Regression"

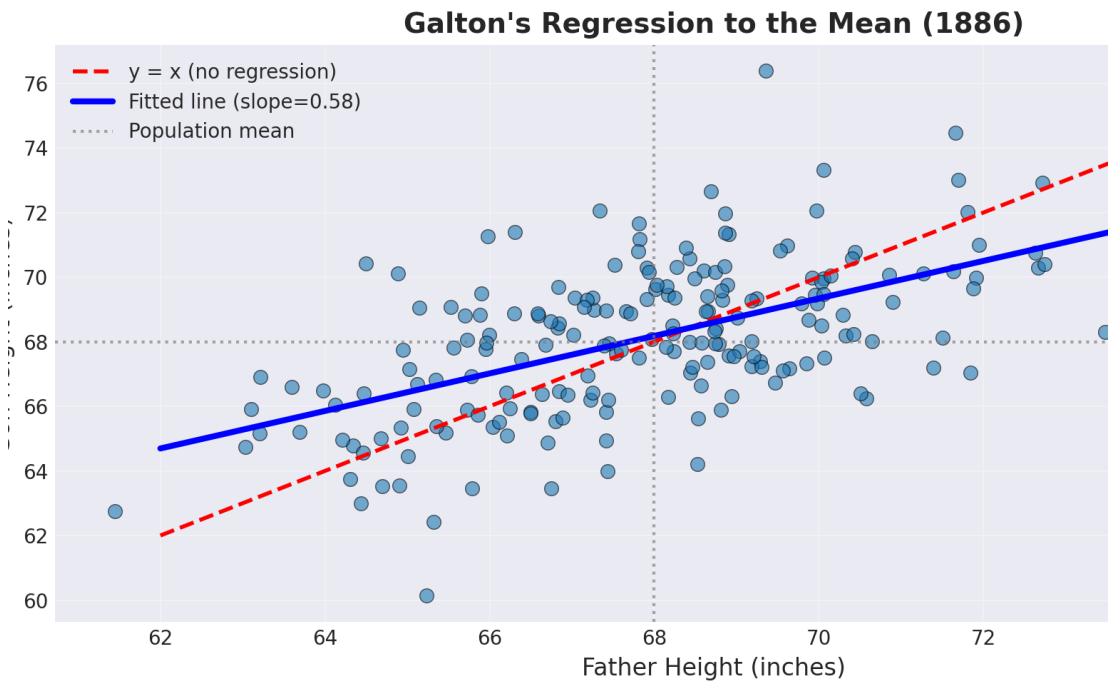
Galton's 1886 Discovery

Francis Galton studied father-son heights and noticed:

- Tall fathers tend to have shorter sons (still tall, but closer to average)
- Short fathers tend to have taller sons (still short, but closer to average)

He called this phenomenon **"regression to the mean"**.

Galton's Data (Simulated)



- Key insight: The fitted line has slope < 1. Extreme values "regress" toward the population average.

Why This Matters for ML

Galton's question:

> "How do we predict one variable from another?"

This is the foundation of **supervised learning**. The term "regression" stuck, and now we use it for any continuous prediction task.

Part 2 — Linear Regression Fundamentals

The Linear Regression Model

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Interpretation:

- β_0: intercept (predicted value when all features = 0)
- β_j: change in $\hat{y}$ when x_j increases by 1 (holding others constant)

Metrics: How Do We Measure "Good"?

Common metrics:

- **MAE** (Mean Absolute Error): average absolute difference (easy to interpret)
- **RMSE** (Root Mean Squared Error): penalizes large errors more
- **R^2**: proportion of variance explained (0 = useless, 1 = perfect)

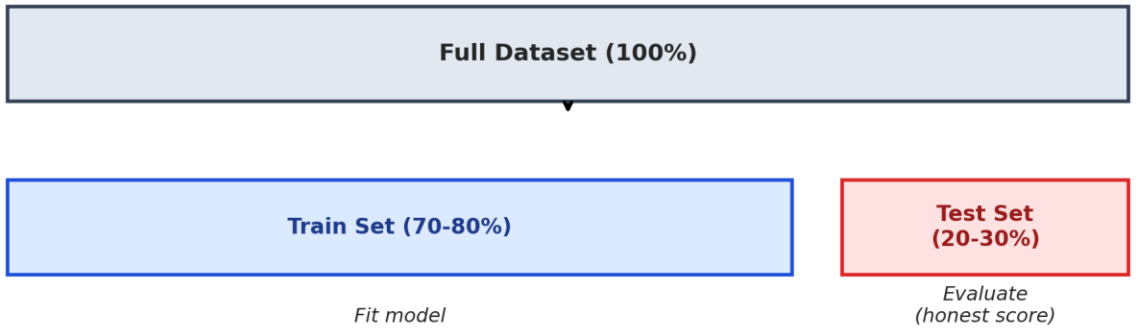
Rule of thumb: report all three.

Train/Test Split: The Honest Evaluation

Why split?

Train set: fit the model

Test set: measure generalization (unseen data)

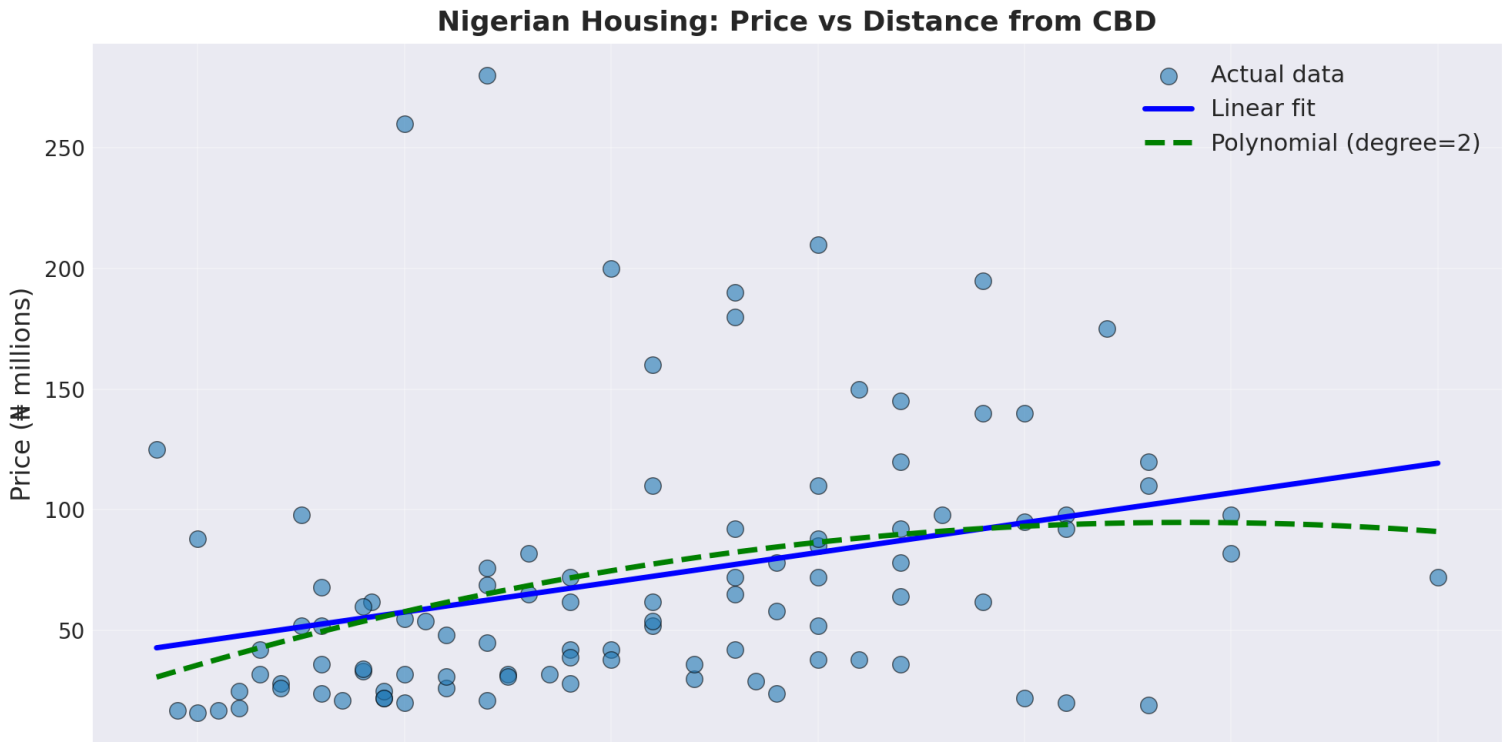


Nigerian Housing Example

Dataset: `nigerian_housing_prices.csv`

- **Numeric:** bedrooms, bathrooms, area_sqm, age_years, distance_cbd_km
- **Categorical:** state, property_type
- **Binary:** has_generator
- **Target:** price_ngn

Visual: Price vs Distance from CBD



Part 3 — Polynomial Features & Overfitting

Why Polynomial Features?

Linear regression fits straight lines. Real relationships are often curved.

Solution: add polynomial terms like x^2, x^3, etc.

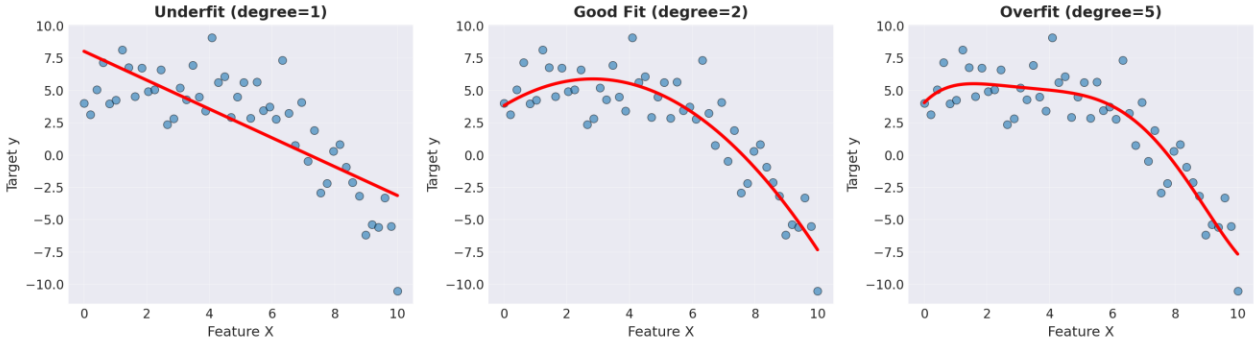
Example: `distance_cbd_km` might have a quadratic relationship with price.

The Danger: Overfitting

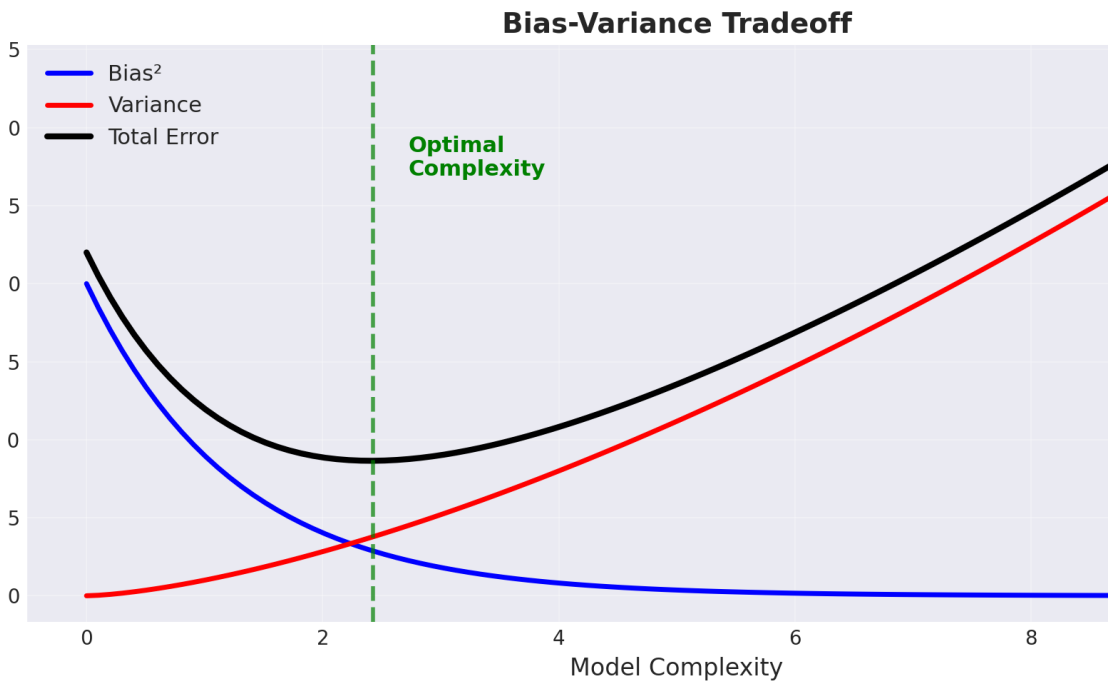
Degree 1: too simple (underfit)

Degree 2: captures the curve (good fit)

Degree 5: follows noise (overfit)



Bias-Variance Tradeoff



- Bias: error from overly simple assumptions

Part 4 — Regularization

What is Regularization?

Regularization **penalizes large coefficients** to prevent overfitting.

Core idea: simpler models (smaller weights) generalize better.

Ridge Regression (L2)

$$\text{Loss} = \text{MSE} + \alpha \sum_{j=1}^{p} \beta_j^2$$

- Shrinks all coefficients smoothly
- Good when you have many correlated features
- **Hyperparameter:** `alpha` (higher α → more shrinkage)

Lasso Regression (L1)

$$\text{Loss} = \text{MSE} + \alpha \sum_{j=1}^{p} |\beta_j|$$

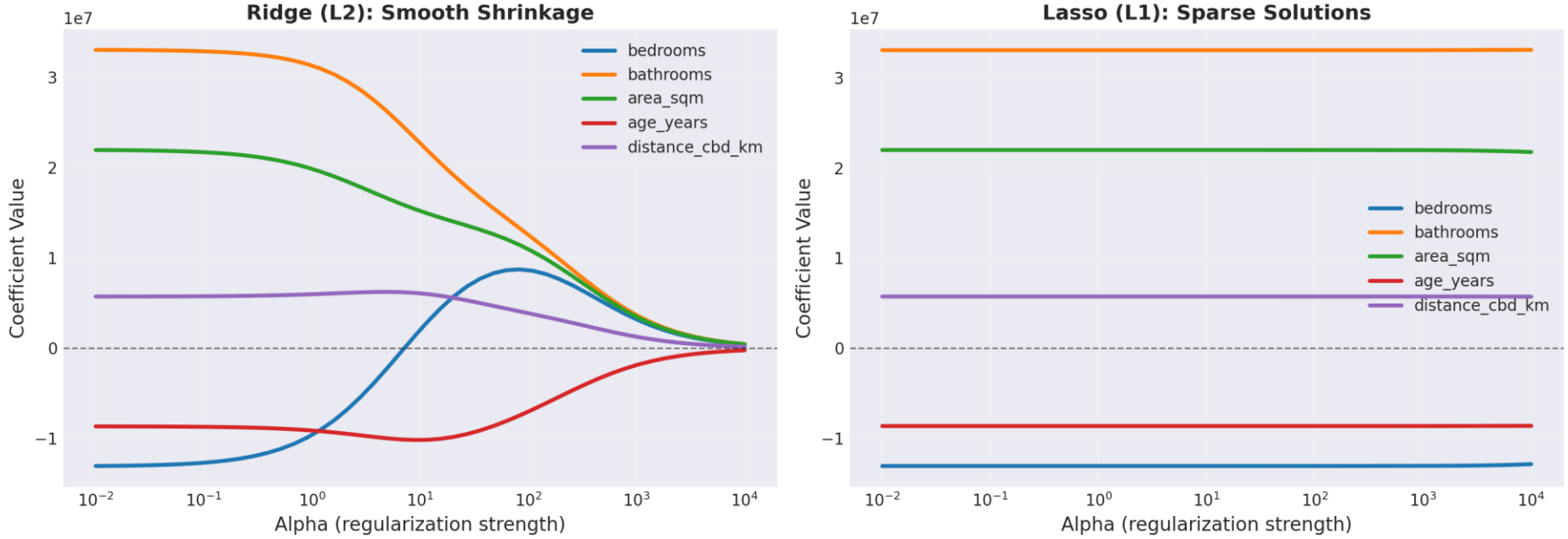
- Can set coefficients **exactly to zero** (sparse solution)
- Acts like automatic feature selection
- **Caution:** can be unstable with highly correlated features

Regularization Paths: Ridge vs Lasso

Observation:

Ridge: coefficients shrink smoothly

Lasso: some coefficients hit zero (sparse)



ElasticNet: Best of Both

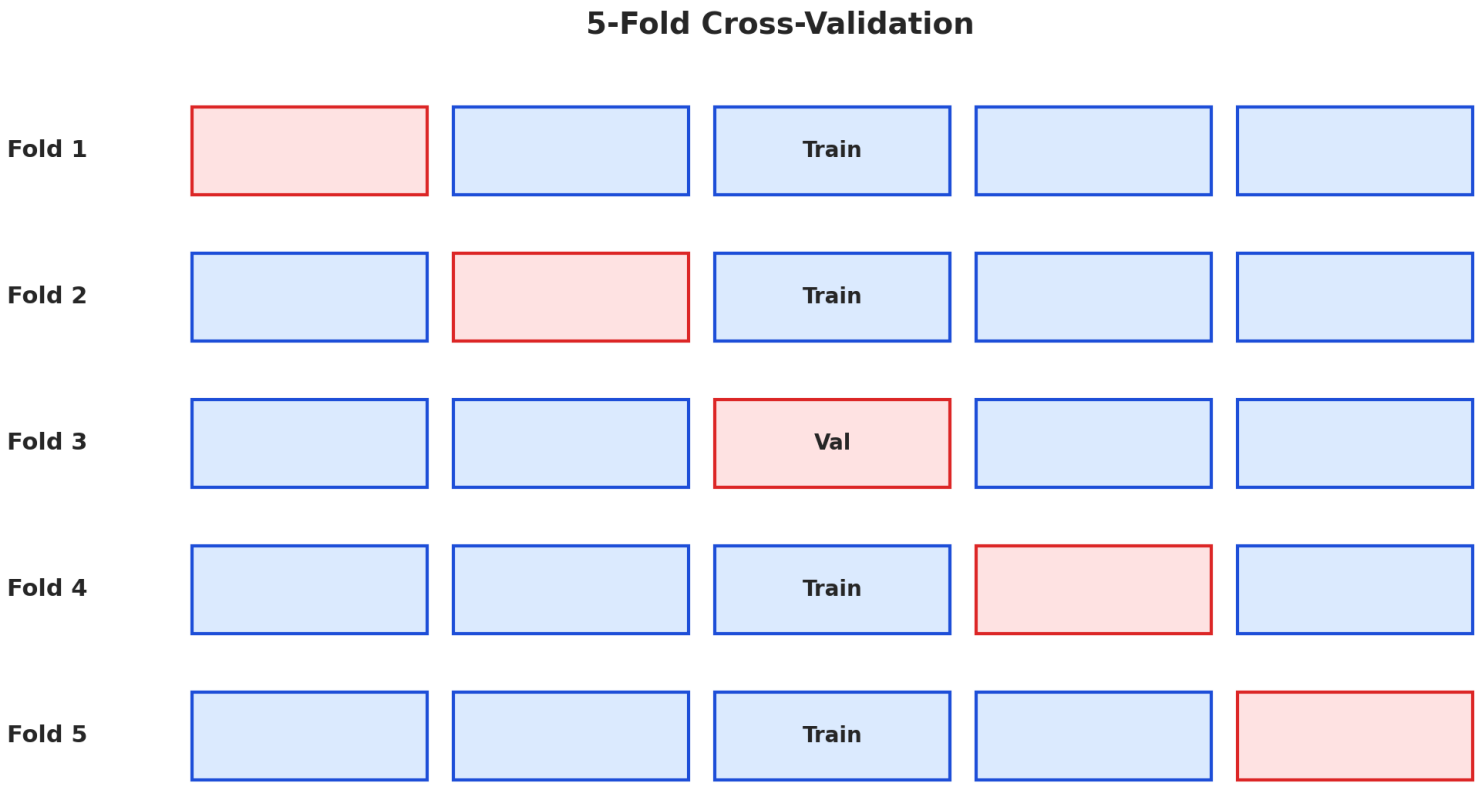
Combines L1 + L2:

$$\text{Loss} = \text{MSE} + \alpha \left(r \sum |\beta_j| + (1 - r) \sum \beta_j^2 \right)$$

- **Hyperparameters:** `alpha`, `l1_ratio`
- Useful when you want selection + stability

Part 5 — Cross-Validation & Tuning

Cross-Validation: Robust Evaluation



Average performance across all folds → robust estimate

Hyperparameter Tuning

GridSearchCV: try all combinations

```
param_grid = {
    'ridge__alpha': [0.01, 0.1, 1, 10, 100]
}
```

RandomizedSearchCV: sample random combinations (faster for large grids)

Best Practices

1. **Tune on train set** (using CV)
2. **Evaluate once on test set**
3. **Always scale features** before regularization
4. **Keep a baseline** (simple model) for comparison
5. **Use pipelines** to prevent leakage

Pipeline Mental Model

```
Raw data

Preprocessing (scaling, encoding)

Feature engineering (polynomial features)

Model (Ridge / Lasso / etc.)

Predictions
```

Pipelines ensure consistent preprocessing across train/test.

Practical Takeaways

When to Use What

Use NumPy/Pandas:

- Data loading, cleaning, EDA

Use Linear Regression:

- Baseline, interpretable coefficients

Use Polynomial Features:

- When you suspect non-linear relationships (but start with degree 2)

Use Ridge:

- Default choice for many correlated features

Use Lasso:

- When you want automatic feature selection

Common Pitfalls

- **Forgetting to scale** before regularization
- **Polynomial explosion** (degree > 3 gets risky)
- **Train/test leakage** (fit scaler on full data)
- **Interpreting coefficients** without checking scale/units

Practice (In Notebooks)

You'll work through:

1. **Live coding:** Baseline linear regression + pipeline
2. **Live coding:** Regularization + tuning with GridSearchCV
3. **Practice:** Hands-on exercises with Nigerian housing data
4. **Assignment:** Build a complete regression pipeline for your project

Assignment Overview

Build a regression model for Nigerian housing prices:

- Load + explore the dataset
- Create a baseline linear regression
- Try polynomial features (degree 2 or 3)
- Apply Ridge/Lasso/ElasticNet
- Tune hyperparameters with CV
- Document metrics + interpret coefficients
- Write 2-3 observations about your findings

Resources

- Galton's original paper (1886): "Regression towards mediocrity in hereditary stature"
- scikit-learn docs: https://scikit-learn.org/stable/
- Nigerian data:
  - National Bureau of Statistics: https://nigerianstat.gov.ng
  - HDX Nigeria: https://data.humdata.org/group/nga

Q&A

Good questions to ask:

- When should I use polynomial features?
- Ridge vs Lasso: how do I choose?
- How do I handle outliers in housing prices?
- How do I interpret coefficients with one-hot encoding?
- What does "alpha" actually do?

Key Takeaways

1. **Regression** comes from Galton's discovery of "regression to the mean"
2. **Bias-variance tradeoff** is central to ML
3. **Regularization** controls overfitting by penalizing large coefficients
4. **Cross-validation** gives robust performance estimates
5. **Pipelines** prevent leakage and ensure reproducibility

Next Week Preview

- Feature engineering deep dive
- Handling missing data & outliers
- Advanced model evaluation (residual plots, learning curves)
- Classification fundamentals (logistic regression)

Keep building! 🚀